Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа №4
По дисциплине «Современный платформы программирования»
Тема: «решении практических задач на Pythone с использованием паттернов»

Выполнил:
Студент 3 курса
Группы ПО-12
Боричевский Д. О.
Вариант 19

Проверила:
Кулик А. Д.

Брест 2026

Цель:

Цель работы: научиться работать с Github API, приобрести практические навыки написания программ для работы с REST API или GraphQL API

Задание 1

Общее задание: используя Github API, реализовать предложенное задание на языке Python. Выполнить визуализацию результатов, с использованием графика или отчета. Можно использовать как REST API (рекомендуется), так и GraphQL

2. Анализ активности разработчиков в open-source проекте

Условие:

Напишите Python-скрипт, который анализирует активность разработчиков в указанном **публичном GitHub-репозитории** и составляет **рейтинг самых активных контрибьюторов**.

1. Запрашивает у пользователя **имя репозитория** (например, pallets/flask или microsoft/vscode).
2. Использует **GitHub API** для получения списка **всех контрибьюторов**.
3. Для каждого контрибьютора собирает данные:
   - Количество коммитов
   - Количество открытых pull requests
   - Количество закрытых pull requests
   - Количество открытых issues
   - Количество закрытых issues
   - Дата последней активности
4. Определяет **самых активных разработчиков** по суммарному вкладу.
5. Строит **график вклада разработчиков** (matplotlib / seaborn).

```
Введите репозиторий для анализа (owner/repo): fastapi/fastapi
Анализируем вклад контрибьюторов в "fastapi/fastapi"...
ТОП-5 самых активных разработчиков:
  1. user1 - 150 коммитов, 40 PR, 20 issues
```

```python
import os
import sys
from datetime import datetime, timedelta
from typing import Dict, List, Optional, Any
from dataclasses import dataclass, asdict
import time

import requests
import matplotlib.pyplot as plt
import seaborn as sns
from matplotlib.patches import Patch
```

```python
# Настройка стилей для графиков
sns.set_style("whitegrid")
plt.rcParams['figure.figsize'] = (14, 10)
plt.rcParams['font.size'] = 10


@dataclass
class ContributorStats:
    """Класс для хранения статистики контрибьютора."""
    username: str
    commits: int = 0
    open_prs: int = 0
    closed_prs: int = 0
    open_issues: int = 0
    closed_issues: int = 0
    last_activity: Optional[datetime] = None

    @property
    def total_prs(self) -> int:
        """Общее количество PR."""
        return self.open_prs + self.closed_prs

    @property
    def total_issues(self) -> int:
        """Общее количество issues."""
        return self.open_issues + self.closed_issues

    @property
    def total_contributions(self) -> int:
        """Суммарный вклад."""
        return self.commits + self.total_prs + self.total_issues

    @property
    def activity_score(self) -> float:
        """
        Расчетный показатель активности.
        Коммиты весят больше, так как это прямые изменения кода.
        """
        return (
                self.commits * 2.0 +  # Коммиты важнее всего
                self.total_prs * 1.5 +  # PR тоже важны
                self.total_issues * 0.5  # Issues менее значимы
        )

    def to_dict(self) -> Dict[str, Any]:
        """Конвертация в словарь."""
        data = asdict(self)
        data['total_prs'] = self.total_prs
        data['total_issues'] = self.total_issues
        data['total_contributions'] = self.total_contributions
        data['activity_score'] = round(self.activity_score, 2)
        data['last_activity'] = (
            self.last_activity.strftime('%Y-%m-%d %H:%M')
            if self.last_activity else 'N/A'
        )
        return data


class GitHubAnalyzer:
    """Класс для анализа GitHub репозитория."""

    BASE_URL = "https://api.github.com"

    def __init__(self, token: Optional[str] = None):
        """
        Инициализация анализатора.

        Args:
            token: GitHub Personal Access Token (опционально, но рекомендуется)
```

```python
        """
        self.session = requests.Session()
        self.session.headers.update({
            "Accept": "application/vnd.github.v3+json",
            "User-Agent": "GitHub-Contributor-Analyzer"
        })

        if token:
            self.session.headers["Authorization"] = f"token {token}"

        self.rate_limit_remaining = None
        self.rate_limit_reset = None

    def _make_request(self, endpoint: str, params: Optional[Dict] = None) -> Dict:
        """
        Выполнение запроса к GitHub API с обработкой rate limiting.

        Args:
            endpoint: API endpoint (без базового URL)
            params: Query parameters

        Returns:
            JSON ответ

        Raises:
```

Вывод:

Я закрепил навыки паттерна программирования на языке Python